JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Interface NavigableSet<E>

Type Parameters:

`E - the type of elements maintained by this set`

All Superinterfaces:

`Collection<E>, Iterable<E>, Set<E>, SortedSet<E>`

All Known Implementing Classes:

`ConcurrentSkipListSet, TreeSet`

```
public interface NavigableSet<E>
extends SortedSet<E>
```

A `SortedSet` extended with navigation methods reporting closest matches for given search targets. Methods `lower`, `floor`, `ceiling`, and `higher` return elements respectively less than, less than or equal, greater than or equal, and greater than a given element, returning `null` if there is no such element. A `NavigableSet` may be accessed and traversed in either ascending or descending order. The `descendingSet` method returns a view of the set with the senses of all relational and directional methods inverted. The performance of ascending operations and views is likely to be faster than that of descending ones. This interface additionally defines methods `pollFirst` and `pollLast` that return and remove the lowest and highest element, if one exists, else returning `null`. Methods `subSet`, `headSet`, and `tailSet` differ from the like-named `SortedSet` methods in accepting additional arguments describing whether lower and upper bounds are inclusive versus exclusive. Subsets of any `NavigableSet` must implement the `NavigableSet` interface.

The return values of navigation methods may be ambiguous in implementations that permit `null` elements. However, even in this case the result can be disambiguated by checking `contains(null)`. To avoid such issues, implementations of this interface are encouraged to *not* permit insertion of `null`

elements. (Note that sorted sets of `Comparable` elements intrinsically do not permit `null`.)

Methods `subSet(E, E)`, `headSet(E)`, and `tailSet(E)` are specified to return `SortedSet` to allow existing implementations of `SortedSet` to be compatibly retrofitted to implement `NavigableSet`, but extensions and implementations of this interface are encouraged to override these methods to return `NavigableSet`.

This interface is a member of the Java Collections Framework.

Since:

`1.6`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
`E`	`ceiling(E e)`	Returns the least element in this set greater than or equal to the given element, or `null` if there is no such element.
`Iterator<E>`	`descendingIterator()`	Returns an iterator over the elements in this set, in descending order.
`NavigableSet<E>`	`descendingSet()`	Returns a reverse order view of the elements contained in this set.
`E`	`floor(E e)`	Returns the greatest element in this set less than or equal to the given element, or `null` if there is no such element.
`SortedSet<E>`	`headSet(E toElement)`	Returns a view of the portion of this set whose elements are strictly less than `toElement`.
`NavigableSet<E>`	`headSet(E toElement, boolean inclusive)`	Returns a view of the portion of this set whose elements are less than (or equal to, if `inclusive` is true) `toElement`.

E	**higher**(**E** e)	Returns the least element in this set strictly greater than the given element, or `null` if there is no such element.
Iterator<**E**>	**iterator**()	Returns an iterator over the elements in this set, in ascending order.
E	**lower**(**E** e)	Returns the greatest element in this set strictly less than the given element, or `null` if there is no such element.
E	**pollFirst**()	Retrieves and removes the first (lowest) element, or returns `null` if this set is empty.
E	**pollLast**()	Retrieves and removes the last (highest) element, or returns `null` if this set is empty.
NavigableSet<**E**>	**subSet**(**E** fromElement, boolean fromInclusive, **E** toElement, boolean toInclusive)	Returns a view of the portion of this set whose elements range from `fromElement` to `toElement`.
SortedSet<**E**>	**subSet**(**E** fromElement, **E** toElement)	Returns a view of the portion of this set whose elements range from `fromElement`, inclusive, to `toElement`, exclusive.
SortedSet<**E**>	**tailSet**(**E** fromElement)	Returns a view of the portion of this set whose elements are greater than or equal to `fromElement`.
NavigableSet<**E**>	**tailSet**(**E** fromElement, boolean inclusive)	Returns a view of the portion of this set whose elements are greater than (or equal to, if `inclusive` is true) `fromElement`.

Methods inherited from interface java.util.SortedSet

comparator, first, last, spliterator

Methods inherited from interface java.util.Set

add, addAll, clear, contains, containsAll, equals, hashCode, isEmpty, remove, removeAll, retainAll, size, toArray, toArray

Methods inherited from interface java.util.Collection

parallelStream, removeIf, stream

Methods inherited from interface java.lang.Iterable

forEach

Method Detail

lower

```
E lower(E e)
```

Returns the greatest element in this set strictly less than the given element, or `null` if there is no such element.

Parameters:

`e - the value to match`

Returns:

`the greatest element less than e, or null if there is no such element`

Throws:

`ClassCastException - if the specified element cannot be compared with the elements currently in the set`

`NullPointerException - if the specified element is null and this set does not permit null elements`

floor

```
E floor(E e)
```

Returns the greatest element in this set less than or equal to the given element, or `null` if there is no such element.

Parameters:

`e - the value to match`

Returns:

`the greatest element less than or equal to e, or null if there is no such element`

Throws:

`ClassCastException - if the specified element cannot be compared with the elements currently in the set`

`NullPointerException - if the specified element is null and this set does not permit null elements`

ceiling

`E ceiling(E e)`

Returns the least element in this set greater than or equal to the given element, or `null` if there is no such element.

Parameters:

`e - the value to match`

Returns:

`the least element greater than or equal to e, or null if there is no such element`

Throws:

`ClassCastException - if the specified element cannot be compared with the elements currently in the set`

`NullPointerException - if the specified element is null and this set does not permit null elements`

higher

`E higher(E e)`

Returns the least element in this set strictly greater than the given element, or `null` if there is no such element.

Parameters:

`e - the value to match`

Returns:

`the least element greater than e, or null if there is no such element`

Throws:

ClassCastException - if the specified element cannot be compared with the elements currently in the set

NullPointerException - if the specified element is null and this set does not permit null elements

pollFirst

E pollFirst()

Retrieves and removes the first (lowest) element, or returns null if this set is empty.

Returns:

the first element, or null if this set is empty

pollLast

E pollLast()

Retrieves and removes the last (highest) element, or returns null if this set is empty.

Returns:

the last element, or null if this set is empty

iterator

Iterator<E> iterator()

Returns an iterator over the elements in this set, in ascending order.

Specified by:

iterator in interface Collection<E>

Specified by:

iterator in interface Iterable<E>

Specified by:

iterator in interface Set<E>

Returns:

an iterator over the elements in this set, in ascending order

descendingSet

```
NavigableSet<E> descendingSet()
```

Returns a reverse order view of the elements contained in this set. The descending set is backed by this set, so changes to the set are reflected in the descending set, and vice-versa. If either set is modified while an iteration over either set is in progress (except through the iterator's own `remove` operation), the results of the iteration are undefined.

The returned set has an ordering equivalent to `Collections.reverseOrder(comparator())`. The expression `s.descendingSet().descendingSet()` returns a view of `s` essentially equivalent to `s`.

Returns:

a reverse order view of this set

descendingIterator

```
Iterator<E> descendingIterator()
```

Returns an iterator over the elements in this set, in descending order. Equivalent in effect to `descendingSet().iterator()`.

Returns:

an iterator over the elements in this set, in descending order

subSet

```
NavigableSet<E> subSet(E fromElement,
                       boolean fromInclusive,
                       E toElement,
                       boolean toInclusive)
```

Returns a view of the portion of this set whose elements range from `fromElement` to `toElement`. If `fromElement` and `toElement` are equal, the returned set is empty unless `fromInclusive` and `toInclusive` are both true. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Parameters:

fromElement - low endpoint of the returned set

fromInclusive - true if the low endpoint is to be included in the returned view

toElement - high endpoint of the returned set

toInclusive - true if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this set whose elements range from fromElement, inclusive, to toElement, exclusive

Throws:

ClassCastException - if fromElement and toElement cannot be compared to one another using this set's comparator (or, if the set has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if fromElement or toElement cannot be compared to elements currently in the set.

NullPointerException - if fromElement or toElement is null and this set does not permit null elements

IllegalArgumentException - if fromElement is greater than toElement; or if this set itself has a restricted range, and fromElement or toElement lies outside the bounds of the range.

headSet

```
NavigableSet<E> headSet(E toElement,
                        boolean inclusive)
```

Returns a view of the portion of this set whose elements are less than (or equal to, if `inclusive` is true) `toElement`. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Parameters:

toElement - high endpoint of the returned set

inclusive - true if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this set whose elements are less than (or equal to, if inclusive is true) toElement

Throws:

ClassCastException - if toElement is not compatible with this set's comparator (or, if the set has no comparator, if toElement

does not implement Comparable). Implementations may, but are not required to, throw this exception if toElement cannot be compared to elements currently in the set.

NullPointerException - if toElement is null and this set does not permit null elements

IllegalArgumentException - if this set itself has a restricted range, and toElement lies outside the bounds of the range

tailSet

```
NavigableSet<E> tailSet(E fromElement,
                        boolean inclusive)
```

Returns a view of the portion of this set whose elements are greater than (or equal to, if `inclusive` is true) `fromElement`. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Parameters:

fromElement - low endpoint of the returned set

inclusive - true if the low endpoint is to be included in the returned view

Returns:

a view of the portion of this set whose elements are greater than or equal to fromElement

Throws:

ClassCastException - if fromElement is not compatible with this set's comparator (or, if the set has no comparator, if fromElement does not implement Comparable). Implementations may, but are not required to, throw this exception if fromElement cannot be compared to elements currently in the set.

NullPointerException - if fromElement is null and this set does not permit null elements

IllegalArgumentException - if this set itself has a restricted range, and fromElement lies outside the bounds of the range

subSet

```
SortedSet<E> subSet(E fromElement,
                    E toElement)
```

Returns a view of the portion of this set whose elements range from `fromElement`, inclusive, to `toElement`, exclusive. (If `fromElement` and `toElement` are equal, the returned set is empty.) The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Equivalent to `subSet(fromElement, true, toElement, false)`.

Specified by:

`subSet` in interface `SortedSet<E>`

Parameters:

`fromElement` - low endpoint (inclusive) of the returned set

`toElement` - high endpoint (exclusive) of the returned set

Returns:

a view of the portion of this set whose elements range from fromElement, inclusive, to toElement, exclusive

Throws:

`ClassCastException` - if fromElement and toElement cannot be compared to one another using this set's comparator (or, if the set has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if fromElement or toElement cannot be compared to elements currently in the set.

`NullPointerException` - if fromElement or toElement is null and this set does not permit null elements

`IllegalArgumentException` - if fromElement is greater than toElement; or if this set itself has a restricted range, and fromElement or toElement lies outside the bounds of the range

headSet

```
SortedSet<E> headSet(E toElement)
```

Returns
a view of the portion of this set whose elements are strictly less than `toElement`. The returned set is backed by this set, so changes in the returned

set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Equivalent to `headSet(toElement, false)`.

Specified by:

`headSet` in interface `SortedSet<E>`

Parameters:

`toElement` - high endpoint (exclusive) of the returned set

Returns:

a view of the portion of this set whose elements are strictly less than toElement

Throws:

`ClassCastException` - if toElement is not compatible with this set's comparator (or, if the set has no comparator, if toElement does not implement `Comparable`). Implementations may, but are not required to, throw this exception if toElement cannot be compared to elements currently in the set.

`NullPointerException` - if toElement is null and this set does not permit null elements

`IllegalArgumentException` - if this set itself has a restricted range, and toElement lies outside the bounds of the range

tailSet

`SortedSet<E> tailSet(E fromElement)`

Returns a view of the portion of this set whose elements are greater than or equal to `fromElement`. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Equivalent to `tailSet(fromElement, true)`.

Specified by:

`tailSet` in interface `SortedSet<E>`

Parameters:

`fromElement` - low endpoint (inclusive) of the returned set

Returns:

a view of the portion of this set whose elements are greater than or equal to fromElement

Throws:

`ClassCastException` - if fromElement is not compatible with this set's comparator (or, if the set has no comparator, if fromElement does not implement `Comparable`). Implementations may, but are not required to, throw this exception if fromElement cannot be compared to elements currently in the set.

`NullPointerException` - if fromElement is null and this set does not permit null elements

`IllegalArgumentException` - if this set itself has a restricted range, and fromElement lies outside the bounds of the range

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.